

COURSE

Основы программирования

JS

Исключения

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Понятие исключения. **3**

Конструкция TRY…CATCH **3**

Генерация ошибок. Оператор throw **4**

Проброс исключений. **5**

Оборачивание исключения. **7**

Литература и другие источники *11*

Вопросы для самопроверки *11*

ПЛАН ЛЕКЦИИ

1. Понятие исключения;
2. Конструкция Try..Catch;
3. Генерация ошибок. Оператор throw;
4. Проброс исключений;
5. Оборачивание исключения.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Понятие исключения.

Обработка исключительных ситуаций (англ. exception handling) — механизм языков программирования, предназначенный для описания реакции программы на ошибки времени выполнения и другие возможные проблемы (исключения), которые могут возникнуть при выполнении программы и приводят к невозможности (бессмысленности) дальнейшей отработки программой её базового алгоритма.

Во время выполнения программы могут возникать ситуации, когда состояние внешних данных, устройств ввода-вывода или компьютерной системы в целом делает дальнейшие вычисления в соответствии с базовым алгоритмом невозможными или бессмысленными. Классические примеры подобных ситуаций приведены ниже.

- ***Целочисленное деление на ноль.*** Конечного результата у данной операции быть не может, поэтому ни дальнейшие вычисления, ни попытка использования результата деления не приведут к решению задачи.
- ***Ошибка при попытке считать данные с внешнего устройства.*** Если данные не удаётся получить, любые дальнейшие запланированные операции с ними бессмысленны.
- ***Исчерпание доступной памяти.*** Если в какой-то момент система оказывается не в состоянии выделить достаточный для прикладной программы объём оперативной памяти, программа не сможет работать нормально.

Конструкция try…catch

Для отслеживания возникновения исключений служит ***конструкция try..catch***, которая состоит из двух основных блоков: try, и затем catch:

```
try {
    // код ...
} catch (err) {
    // обработка ошибки
}
```

Работает она так. Код выполняется внутри блока try. Если в нём ошибок нет, то блок catch(err) игнорируется, то есть выполнение доходит до конца try и потом прыгает через catch. Если в нём возникнет ошибка, то выполнение try на ней прерывается, и управление прыгает в начало блока catch(err). При этом переменная err (можно выбрать и другое название) будет содержать объект ошибки с подробной информацией о произошедшем.

Таким образом, при ошибке в try скрипт не «падает», и мы получаем возможность обработать ошибку внутри catch.

Рассмотрим несколько примеров.

Пример без ошибок: при запуске сработают alert (1) и (2):

```
try {
        alert('Начало блока try');  // (1)
        // .. код без ошибок
        alert('Конец блока try');   // (2)
} catch(e) {
        alert('Блок catch не получит управление, так как нет ошибок'); // (3)
}
alert("Потом код продолжит выполнение...");
```

Пример с ошибкой: при запуске сработают (1) и (3):

```
try {
        alert('Начало блока try');  // (1)
        lalala; // ошибка, переменная не определена!
        alert('Конец блока try');  // (2)
} catch(e) {
        alert('Ошибка ' + e.name + ":" + e.message + "\n" + e.stack); // (3) <--
}
alert("Потом код продолжит выполнение...");
```

Если грубо нарушена структура кода, например, не закрыта фигурная скобка или где-то стоит лишняя запятая, то никакой try..catch здесь не поможет. Такие ошибки называются синтаксическими, интерпретатор не может понять такой код.

Ошибку, которая произойдёт в коде, запланированном «на будущее», например, в setTimeout, try..catch не поймает. Например,

```
try {
        setTimeout(function() {
        throw new Error(); // вылетит в консоль
  }, 1000);
} catch (e) {
        alert( "не сработает" );
}
```

На момент запуска функции, назначенной через setTimeout, этот код уже завершится, интерпретатор выйдет из блока try..catch. Чтобы поймать ошибку внутри функции из setTimeout, и try..catch должен быть в той же функции.

Генерация ошибок. Оператор throw.

Представим ситуацию, что, например, полученные данные являются корректным JSON-объектом. Но в этом объекте нет нужного свойства name:

```
var data = '{ "age": 30 }'; // данные неполные
try {
```

```
        var user = JSON.parse(data); // <-- выполнится без ошибок
        alert( user.name ); // undefined
} catch (e) {
        // не выполнится
        alert( "Извините, в данных ошибка" );
}
```

Вызов JSON.parse выполнится без ошибок, но ошибка в данных есть. И, так как свойство name обязательно должно быть, то для нас это такие же некорректные данные, как и "Has Error". Для того, чтобы унифицировать и объединить обработку ошибок парсинга и ошибок в структуре, воспользуемся оператором throw.

Оператор throw генерирует ошибку и записывается в виде throw <объект ошибки>. Технически в качестве объекта ошибки можно передать что угодно, это может быть даже не объект, а число или строка, но всё же лучше, чтобы это был объект, желательно – совместимый со стандартным, то есть чтобы у него были как минимум свойства name и message.

В качестве конструктора ошибок можно использовать встроенный конструктор: new Error(message) или любой другой.

В JavaScript встроен ряд конструкторов для стандартных ошибок: SyntaxError, ReferenceError, RangeError и некоторые другие. Можно использовать и их, но только чтобы не было путаницы.

В данном случае мы используем конструктор new SyntaxError(message). Он создаёт ошибку того же типа, что и JSON.parse.

```
var data = '{ "age": 30 }'; // данные неполны
try {
        var user = JSON.parse(data); // <-- выполнится без ошибок
        if (!user.name) {
                throw new SyntaxError("Данные некорректны");
        }
        alert( user.name );
} catch (e) {
        alert( "Извините, в данных ошибка" );
}
```

Получилось, что блок catch – единое место для обработки ошибок во всех случаях: когда ошибка выявляется при JSON.parse или позже.

Проброс исключений.

Ранее был рассмотрен пример возникновения ошибки при некорректных данных. Но может ли быть так, что возникнет какая-то другая ошибка? Конечно, может! Код – это вообще мешок с ошибками.

Блок catch в нашем примере предназначен для обработки ошибок, возникающих при некорректных данных. Если же в него попала какая-то другая ошибка, то вывод сообщения о «некорректных данных» будет дезинформацией посетителя. Ошибку, о которой catch не знает, он не должен обрабатывать.

Такая техника называется ***«проброс исключения»***: в catch(e) мы анализируем объект ошибки, и если он нам не подходит, то делаем throw e.

При этом ошибка «выпадает» из try..catch наружу. Далее она может быть поймана либо внешним блоком try..catch (если есть), либо «повалит» скрипт.

В примере ниже catch обрабатывает только ошибки SyntaxError, а остальные – выбрасывает дальше:

```
var data = '{ "name": "Вася", "age": 30 }'; // данные корректны
try {
	var user = JSON.parse(data);
	if (!user.name) {
		throw new SyntaxError("Ошибка в данных");
	}
	blabla(); // произошла непредусмотренная ошибка
	alert( user.name );
} catch (e) {
	if (e.name == "SyntaxError") {
		alert( "Извините, в данных ошибка" );
	} else {
		throw e;
	}
}
```

Заметим, что ошибка, которая возникла внутри блока catch, «выпадает» наружу, как если бы была в обычном коде.

В следующем примере такие ошибки обрабатываются ещё одним, «более внешним» try..catch:

```
function readData() {
	var data = '{ "name": "Вася", "age": 30 }';
	try {
		// ...
		blabla(); // ошибка!
	} catch (e) {
		// ...
		if (e.name != 'SyntaxError') {
			throw e; // пробрасываем
		}
	}
}

try {
	readData();
} catch (e) {
	alert( "Поймал во внешнем catch: " + e ); // ловим
}
```

В примере выше try..catch внутри readData умеет обрабатывать только SyntaxError, а внешний – все ошибки. Без внешнего проброшенная ошибка «вывалилась» бы в консоль с остановкой скрипта.

Оборачивание исключения.

Последняя, самая продвинутая техника по работе с ошибками – это ***оборачивание исключений***. Она, впрочем, является стандартной практикой во многих объектно-ориентированных языках.

Цель некоторой функции readData – прочитать данные. При чтении могут возникать разные ошибки, не только SyntaxError, но и, возможно, к примеру, URIError (неправильное применение функций работы с URI) да и другие. В коде с вызовом readData хотелось бы иметь какой-либо результат, или информацию об ошибке. При этом очень важным является вопрос: обязан ли этот внешний код знать о всевозможных типах ошибок, которые могут возникать при чтении данных, и уметь перехватывать их?

Обычно внешний код может работать «на уровень выше», и получать либо результат, либо «ошибку чтения данных». При этом какая именно ошибка произошла – ему неважно. А если эту ошибку потребуется обработать, то хотелось бы иметь возможность знать ее тип. ***Это важнейший общий подход к проектированию – каждый участок функционала должен получать информацию на том уровне, который ему необходим***.

В данном случае, если при чтении данных происходит ошибка, то она должна генерироваться в виде объекта ReadError, с соответствующим сообщением. А «исходную» ошибку на всякий случай можно сохранить, присвоив его свойство cause (англ. – причина). Выглядит это следующим образом:

```
function ReadError(message, cause) {
	this.message = message;
	this.cause = cause;
	this.name = 'ReadError';
	this.stack = cause.stack;
}

function readData() {
	var data = '{ bad data }';
	try {
		// ...
		JSON.parse(data);
		// ...
	} catch (e) {
		// ...
		if (e.name == 'URIError') {
			throw new ReadError("Ошибка в URI", e);
		} else if (e.name == 'SyntaxError') {
			throw new ReadError("Синтаксическая ошибка в данных", e);
		} else {
			throw e; // пробрасываем
		}
	}
```

```
}

try {
        readData();
} catch (e) {
        if (e.name == 'ReadError') {
                alert( e.message );
                alert( e.cause ); // оригинальная ошибка-причина
        } else {
                throw e;
        }
}
```

Этот подход называют ***«оборачиванием» исключения***, поскольку мы берём ошибки «более низкого уровня» и «заворачиваем» их в ReadError, которая соответствует текущей задаче.

Секция finally

Конструкция try..catch может содержать ещё один блок: finally. Выглядит этот расширенный синтаксис так:

```
try {
        .. пробуем выполнить код ..
} catch(e) {
        .. перехватываем исключение ..
} finally {
        .. выполняем всегда ..
}
```

Секция finally не обязательна, но если она есть, то она выполняется всегда: после блока try, если ошибок не было и после catch, если они были.

Рассмотрим следующий пример.

```
try {
        alert( 'try' );
        if (confirm('Сгенерировать ошибку?')) BAD_CODE();
} catch (e) {
        alert( 'catch' );
} finally {
        alert( 'finally' );
}
```

У него два варианта работы. Если вы ответите на вопрос «сгенерировать ошибку?» - утвердительно, то срабатывает try -> catch -> finally. Если ответите отрицательно, то срабатывает try -> finally.

Секцию finally используют, чтобы завершить начатые операции при любом варианте развития событий. Например, мы хотим подсчитать время на выполнение функции sum(n), которая должна возвратить сумму чисел от 1 до n и работает рекурсивно:

```
function sum(n) {
	return n ? (n + sum(n - 1)) : 0;
}
var n = +prompt('Введите n?', 100);
var start = new Date();
try {
	var result = sum(n);
} catch (e) {
	result = 0;
} finally {
	var diff = new Date() - start;
}
alert( result ? result : 'была ошибка' );
alert( "Выполнение заняло " + diff );
```

Здесь секция finally гарантирует, что время будет подсчитано в любых ситуациях: при ошибке в sum или без неё. Вы можете проверить это, запустив код с указанием n=100 – будет без ошибки, finally выполнится после try, а затем с n=100000 – будет ошибка из-за слишком глубокой рекурсии, управление прыгнет в finally после catch.

Блок finally срабатывает при любом выходе из try..catch, в том числе и return. В примере ниже из try происходит return, но finally получает управление до того, как контроль возвращается во внешний код.

```
function func() {
	try {
		// сразу вернуть значение
		return 1;
	} catch (e) {
		/* ... */
	} finally {
		alert( 'finally' );
	}
}
alert( func() ); // сначала finally, потом 1
```

Если внутри try были начаты какие-то процессы, которые нужно завершить по окончании работы, то в finally это обязательно будет сделано. Кстати, для таких случаев иногда используют try..finally вообще без catch:

```
function func() {
	try {
		return 1;
```

```
        } finally {
                alert( 'Вызов завершён' );
        }
}
alert( func() ); // сначала finally, потом 1
```

В примере выше try..finally вообще не обрабатывает ошибки. Задача в другом: выполнить код при любом выходе из try – с ошибкой ли, без ошибок или через return.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Для закрепления знаний по данной теме можно посмотреть печатные издания, в который рассматриваются концепции создания «совершенного» программного кода:

1. Мартин Р. Чистый код. Создание, анализ и рефакторинг. Библиотека программиста. – СПб.: Питер, 2014. – 464 с.
2. Макконнелл С. Совершенный код. Мастер#класс / Пер. с англ. - М. : Издательство «Русская редакция», 2010. - 896 стр. : ил.

Статьи с обсуждением типовых ошибок в программном коде:

3. Статья «10 самых распространённых ошибок при программировании на JavaScript» (https://habrahabr.ru/company/mailru/blog/233553/).
4. Статья «11 ошибок при работе с JavaScript» (http://codeharmony.ru/materials/132).
5. Серия статей «10 типичных ошибок Node.js разработчиков» (https://makeomatic.ru/blog/2015/04/28/10_mistakes_node/).

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

6. Для чего служит механизм обработки исключительных ситуаций?
7. Как работает конструкция Try … catch? Приведите пример.
8. В каком случае конструкция Try … catch может не отследить ошибку?
9. В каком случает применяется генерация собственного исключения?
10. Как работает оператор trow? Приведите пример.
11. Что такое «проброс исключения»? Приведите пример кода.
12. Какую функцию выполняет секция Finally в конструкции Try…catch?